Lab 1 - Turtle Catcher (nodes, topics, services, parameters, launch)

Aim. Build a game in which one node keeps dropping new turtles into the turtlesim window and another node drives `turtle1` to the nearest one and "catches" it. Start the nodes one at a time, inspect every ROS 2 communication type involved, catch turtles by hand through a service, then start everything with one launch file and change the game with a parameter.

Time. 45 min. **Needs.** Only this folder, plus a C++ compiler (Visual Studio Build Tools with the C++ workload and a Windows SDK) because the custom messages are compiled.

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (`winget install --id prefix-dev.pixi -e --source winget`), and Visual Studio Build Tools (C++ workload) and a Windows 10/11 SDK, for the compiled message package.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

ROS 2 idea	Where it appears in this lab
Node - one program, one job	`turtlesim` (the window), `turtle_spawner`, `turtle_controller`
Topic - continuous one-way data	`/turtle1/pose` (where is it), `/turtle1/cmd_vel` (how to move), `/alive_turtles` (who is on screen)
Service - one request, one answer	`/spawn`, `/kill` (turtlesim) and our own `/catch_turtle`
Parameter - a setting read at start-up	`spawn_frequency`, `catch_closest_turtle_first`
Custom interface - your own message / service types	`Turtle.msg`, `TurtleArray.msg`, `CatchTurtle.srv` in `turtle_catcher_interfaces`

ROS 2 idea	Where it appears in this lab
Launch file - starts several nodes together	`turtle_catcher.launch.py`

```
turtlesim --/turtle1/pose--> turtle_controller --/turtle1/cmd_vel--> turtlesim
turtle_spawner --/spawn--> turtlesim           turtle_controller --/catch_turtle--> turtle_spawner
--/kill--> turtlesim
turtle_spawner --/alive_turtles--> turtle_controller
```

Control law (`logic.py`): with distance `d` and heading error `e` to the target, `v = min(2 d, 3)` and `w = 6 e`; if `|e| > 60 deg` the turtle first turns on the spot (`v = 0`). A turtle is caught when `d < 0.5`.

Procedure

1. Build (also needed after every edit in `ws\src`).

```
pixi run build
```

2. Look at the custom interface. A service has a request part and a reply part, separated by `---`.

```
pixi run ros interface show turtle_catcher_interfaces/srv/CatchTurtle
pixi run ros interface show turtle_catcher_interfaces/msg/TurtleArray
```

Part A - the nodes one at a time

3. T1 - the window. T2 - the spawner. A turtlesim window opens; from now on a new turtle appears every second and nobody removes it.

```
pixi run ros run turtlesim turtlesim_node
```

```
pixi run ros run turtle_catcher turtle_spawner
```

4. T3 - inspect.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros topic list --no-daemon --spin-time 5
```

The spawner publishes the list of turtles on the screen (names `turtleN`; the highest N tells how many were spawned) and reads its settings from parameters:

```
pixi run ros topic echo /alive_turtles --no-daemon --once --spin-time 5
pixi run ros param get /turtle_spawner spawn_frequency --no-daemon --spin-time 5
```

5. T3 - call the service by hand. An unknown name must be refused (`success=False`). Then catch five turtles from the list yourself: each call that names a living turtle answers `success=True` and the turtle disappears from the window.

```
pixi run ros service call /catch_turtle turtle_catcher_interfaces/srv/CatchTurtle "{name: nope}"
$names = (pixi run ros topic echo /alive_turtles --no-daemon --once --spin-time 5 | Select-String
'name: (\w+)').Matches | ForEach-Object { $_.Groups[1].Value } | Select-Object -First 5
foreach ($n in $names) { "$n  " + (pixi run ros service call /catch_turtle
turtle_catcher_interfaces/srv/CatchTurtle "{name: $n}" | Select-String 'success=\w+').Matches.Value
}
```

Part B - the whole game from the launch file

6. Stop T1 and T2 (Ctrl+C), then T1 - start all three nodes (turtlesim, spawner, controller) with the settings from `config\turtle_catcher.yaml`. Now `turtle1` chases the nearest turtle and catches it by calling the same service you just used.

```
# Ctrl+C in T1 and T2
```

```
pixi run ros launch turtle_catcher turtle_catcher.launch.py
```

7. T3 - how many turtles are alive? This line waits 60 s, then prints how many turtles are alive and how many were spawned so far.

```
Start-Sleep 60
$t = pixi run ros topic echo /alive_turtles --no-daemon --once --spin-time 5 | Out-String; $m =
[regex]::Matches($t,'name: turtle(\d+)'); "alive: $($m.Count)   spawned: $(($m | ForEach-Object
{[int]$_.Groups[1].Value} | Measure-Object -Maximum).Maximum)"
```

8. Experiment: spawn faster. Stop T1 (Ctrl+C). Set `spawn_frequency` to 3.0 in the YAML file, rebuild (the installed copy is what runs), relaunch, wait 60 s, measure again with the line of step 7.

```
# Ctrl+C in T1
```

```
(Get-Content ws\src\turtle_catcher\config\turtle_catcher.yaml) -replace 'spawn_frequency:
1.0','spawn_frequency: 3.0' | Set-Content ws\src\turtle_catcher\config\turtle_catcher.yaml
pixi run build
```

```
pixi run ros launch turtle_catcher turtle_catcher.launch.py
```

```
Start-Sleep 60
$t = pixi run ros topic echo /alive_turtles --no-daemon --once --spin-time 5 | Out-String; $m =
[regex]::Matches($t,'name: turtle(\d+)'); "alive: $($m.Count)   spawned: $(($m | ForEach-Object
{[int]$_.Groups[1].Value} | Measure-Object -Maximum).Maximum)"
```

Put the file back when you are done (Ctrl+C in T1 first):

```
# Ctrl+C in T1
```

```
(Get-Content ws\src\turtle_catcher\config\turtle_catcher.yaml) -replace 'spawn_frequency: 3.0','spawn_frequency: 1.0' | Set-Content ws\src\turtle_catcher\config\turtle_catcher.yaml
pixi run build
```

Observations (a recorded run; yours differ slightly because spawn positions are random)

`spawn_frequency`	turtles alive 60 s after the launch	what it means
1.0 (default)	4-8 (recorded: 4 alive, 70 spawned)	the controller catches turtles as fast as they appear (about 1 per second)
3.0	about 50-60 (recorded: 61 alive, 212 spawned)	spawning outpaces catching, so the screen fills up

In Part A (no controller) every `catch_turtle` call with a listed name returned `success=True`; with `nope` it returned `success=False`.

Verification

```
pixi run test-class1
```

It starts the real launch file (turtlesim offscreen, no window) for 60 s and requires: the stack comes up, an unknown name is rejected, at least 30 turtles spawned, at least 5 caught, fewer than 15 alive, and `turtle1` travelled more than 10 units. Recorded: 6/6 checks (61 spawned, 54 caught, at most 9 alive, 134 units travelled).

Questions

1. Which topic would you echo to see where `turtle1` is? Which one to make it move? Who publishes each?
2. Why is `/catch_turtle` a service and `/alive_turtles` a topic?
3. In `logic.py`, why is the linear speed set to 0 when the heading error exceeds 60 degrees?
4. At `spawn_frequency: 3.0` the alive list grows. Which parameter or constant would you change so the controller keeps up?
5. *(Extension)* Add a `catch_count` topic that the controller publishes after every catch, and watch it with `ros topic echo`.

If it does not work

Symptom	Fix
`pixi run build` fails with `rc.exe` / `kernel32.lib` not found	install the Windows SDK (see "This folder is a complete project")
`package 'turtle_catcher' not found`	run `pixi run build`
`ros topic echo` says "does not appear to be published yet"	add `--spin-time 5`
No turtlesim window appears	wait 30 s on a cold start; `ros node list` should show `/turtlesim`
`success=False` for a name from the list in Part B	the controller caught it first - that is the game working